

Q1. Query Execution Plan and Heuristic Optimization

Question: Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Example Query

Consider the following query:

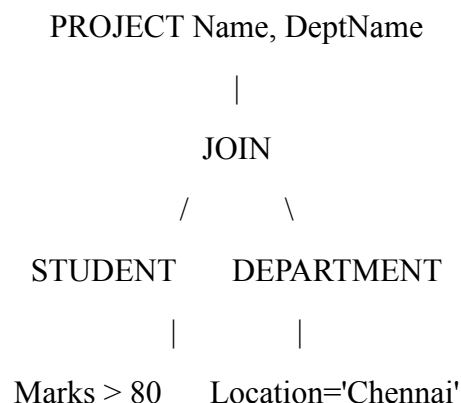
```
SELECT S.Name, D.DeptName
FROM STUDENT S
JOIN DEPARTMENT D ON S.DeptID = D.DeptID
WHERE S.Marks > 80
AND S.DeptID IN
  (SELECT DeptID
   FROM DEPARTMENT
   WHERE Location = 'Chennai');
```

Step 1: Initial Query Tree

The original query contains:

- A selection on Marks > 80
- A sub-query on Location = 'Chennai'
- A join between STUDENT and DEPARTMENT

Basic execution plan:



Step 2: Push Down Selection

Apply the **selection as early as possible**.

Instead of joining all student records first, filter students using:

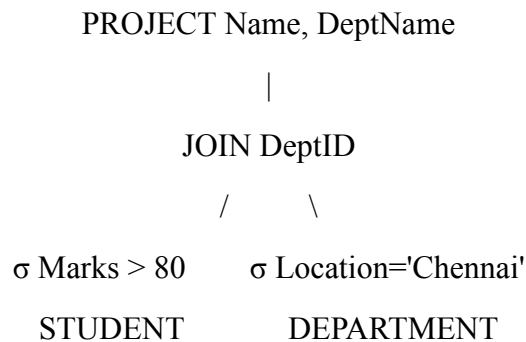
σ Marks > 80 (STUDENT)

Also filter the department table using:

σ Location = 'Chennai' (DEPARTMENT)

Step 3: Perform the Join

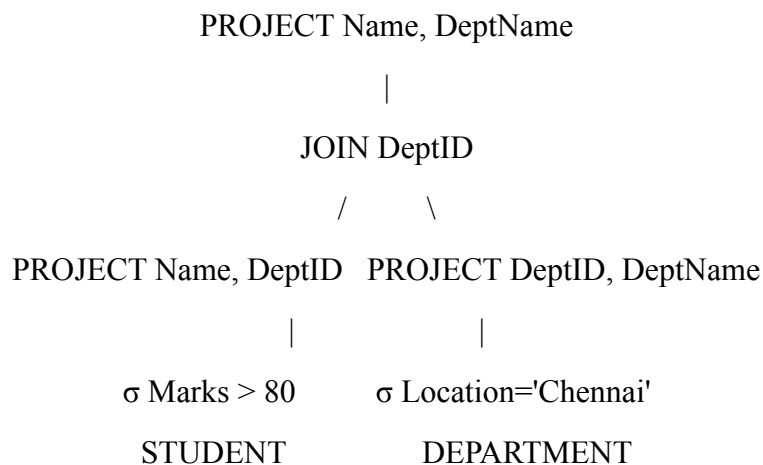
Now join only the filtered records:



This reduces the number of tuples involved in the join.

Step 4: Push Down Projection

Select only the required columns before performing the join.

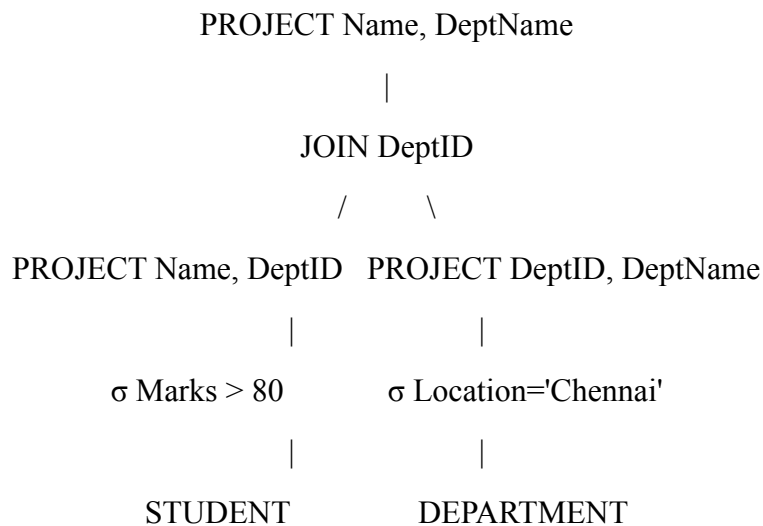


This reduces the amount of data transferred between operations.

Heuristic Rules Used

1. **Push selections down** – Apply WHERE conditions as early as possible.
2. **Push projections down** – Keep only required columns.
3. **Perform selective operations first** – Reduce the number of tuples before joins.
4. **Order joins efficiently** – Perform joins that produce smaller intermediate results first.

Final Optimized Plan



Conclusion

Heuristic query optimization improves query performance by **reducing intermediate results, applying selections and projections early, and arranging joins efficiently**. This decreases processing time and memory usage.

Q2. Cost-Based Query Optimization

Question: Apply cost-based optimization to choose between **Nested-Loop Join, Sort-Merge Join, and Hash Join** for given relations with specified tuple counts and page sizes.

Since the question in the PDF does **not provide actual tuple counts or page sizes**, we can use a simple example to demonstrate the calculation.

Given

Suppose:

- Relation **R** = 1,000 tuples, 100 pages
- Relation **S** = 5,000 tuples, 500 pages
- Available memory = 10 pages

We compare the three join methods.

1. Nested-Loop Join

For page-based nested-loop join:

$$\text{Cost} \approx B(R) + B(R) \times B(S)$$

Using R as the outer relation:

$$\text{Cost} = 100 + (100 \times 500)$$

$$= 50,100 \text{ page I/Os}$$

So, the cost is **50,100 I/Os**.

2. Sort-Merge Join

Sort-merge join first sorts both relations and then merges them.

Its cost is approximately:

$$\text{Cost} \approx \text{Sort}(R) + \text{Sort}(S) + B(R) + B(S)$$

For these relations, sorting requires considerably fewer I/Os than repeatedly scanning the inner relation in nested-loop join.

Therefore:

Sort-Merge Join is much better than Nested-Loop Join when the relations are large and sorting is affordable.

3. Hash Join

Hash join partitions both relations using a hash function and then matches the corresponding partitions.

Approximate cost:

$$\begin{aligned}\text{Cost} &\approx 3 \times (B(R) + B(S)) \\ &= 3 \times (100 + 500) \\ &= 1,800 \text{ page I/Os}\end{aligned}$$

So, the approximate cost is **1,800 I/Os**.

Comparison

Join Method	Approximate Cost	Result
Nested-Loop Join	50,100 I/Os	Highest
Sort-Merge Join	Lower than Nested-Loop	Good
Hash Join	1,800 I/Os	Best

Final Answer

Among the three methods, **Hash Join** is the best choice for this example because it has the lowest estimated I/O cost.

Selection:

Hash Join → Best

Sort-Merge Join → Second choice

Nested-Loop Join → Least suitable

Conclusion

Cost-based optimization selects the join algorithm with the **lowest estimated execution cost**. The decision depends mainly on **tuple count, page size, available memory, and relation size**.

Q3. JDBC Connectivity with MySQL

The question asks to **implement JDBC connectivity in Java with a MySQL database and execute parameterized queries using PreparedStatement.**

Simple Java Program

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {

        try {

            // 1. Establish connection

            Connection con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/college",

                "root",

                ""

            );

            // 2. Create parameterized query

            String sql = "SELECT * FROM STUDENT WHERE DeptID = ?";

            PreparedStatement ps = con.prepareStatement(sql);

            // 3. Set parameter

            ps.setInt(1, 10);

            // 4. Execute query

            ResultSet rs = ps.executeQuery();

            // 5. Display result

            while (rs.next()) {

                System.out.println(
```

```

        rs.getInt("StudentID") + " " +
        rs.getString("Name")
    );
}

// 6. Close connection
rs.close();
ps.close();
con.close();

} catch (SQLException e) {
    System.out.println("Database Error: " + e.getMessage());
}
}
}

```

Explanation

1. Connection

Connection con = DriverManager.getConnection(...);

Connects the Java program to the MySQL database.

2. PreparedStatement

String sql = "SELECT * FROM STUDENT WHERE DeptID = ?";

PreparedStatement ps = con.prepareStatement(sql);

? is a **parameter placeholder**.

3. Set the Parameter

ps.setInt(1, 10);

The first ? is replaced with the value 10.

4. Execute Query

ResultSet rs = ps.executeQuery();

Executes the SQL query and returns the result.

5. Display Data

```
while (rs.next()) {  
    System.out.println(rs.getInt("StudentID"));  
}
```

Reads each returned record.

6. Close Resources

The ResultSet, PreparedStatement, and database connection are closed after use.

Advantages of PreparedStatement

- Executes **parameterized SQL queries**.
- Helps prevent **SQL injection**.
- Can be reused with different parameter values.
- Makes SQL code easier to manage.

Result

Thus, JDBC connectivity between **Java and MySQL** is established successfully, and a parameterized query is executed using PreparedStatement.

Q4. Conflict Serializability Using Precedence Graph

The question asks us to determine whether a schedule of **four concurrent transactions** is conflict-serializable using a **precedence graph**.

Since the PDF does not provide a specific schedule, here is a simple example.

Given Schedule

Consider four transactions **T1, T2, T3, and T4**:

$S = R1(A), W2(A), R3(B), W1(B), R4(A), W3(B)$

Step 1: Identify Conflicts

A conflict occurs when:

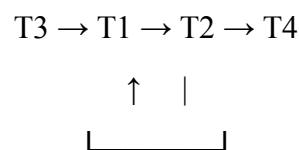
- Operations belong to different transactions.
- They access the same data item.
- At least one operation is a **write**.

Conflicts:

Conflict	Edge
$R1(A) \rightarrow W2(A)$	$T1 \rightarrow T2$
$W2(A) \rightarrow R4(A)$	$T2 \rightarrow T4$
$R3(B) \rightarrow W1(B)$	$T3 \rightarrow T1$
$W1(B) \rightarrow W3(B)$	$T1 \rightarrow T3$

Step 2: Construct Precedence Graph

The graph contains:



There is a cycle:

$T1 \rightarrow T3 \rightarrow T1$

Step 3: Check for Cycle

A precedence graph is **conflict-serializable** only if it has no cycle.

Here, we have:

$T1 \rightarrow T3$

$T3 \rightarrow T1$

Therefore, the graph contains a **cycle**.

Final Answer

The given schedule is NOT conflict-serializable because its precedence graph contains a cycle.

Easy Rule to Remember

No cycle $\rightarrow$ Conflict Serializable ✓

Cycle present $\rightarrow$ Not Conflict Serializable ✗

Conclusion

The precedence graph is used to determine whether a concurrent transaction schedule can be converted into an equivalent serial schedule. If the graph is **acyclic**, a serial order can be obtained using topological sorting.

Q5. Basic Two-Phase Locking (2PL)

The question asks us to **demonstrate Basic Two-Phase Locking on a given transaction set and identify and resolve deadlocks.**

Example

Consider two transactions:

T1: Read(A) → Write(A) → Read(B)

T2: Read(B) → Write(B) → Read(A)

Step 1: Two Phases of 2PL

Basic 2PL has **two phases**:

1. **Growing Phase** – Transaction can acquire locks but cannot release locks.
2. **Shrinking Phase** – Transaction can release locks but cannot acquire new locks.

Step 2: Apply Locks

Initially:

T1 → Lock-X(A)

T2 → Lock-X(B)

Now:

T1 holds X(A)

T2 holds X(B)

Next:

T1 requests X(B) → WAIT

T2 requests X(A) → WAIT

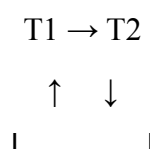
Step 3: Deadlock Occurs

The situation becomes:

T1 → waiting for B → T2

T2 → waiting for A → T1

Therefore:



There is a **cycle**, so a **deadlock** has occurred.

Step 4: Resolve the Deadlock

One transaction can be aborted/rolled back.

For example:

Abort T2

↓

Release X(B)

↓

T1 obtains X(B)

↓

T1 completes

Then T2 can be restarted later.

Final Sequence

T1: Lock-X(A)

T2: Lock-X(B)

T1: Request X(B) → WAIT

T2: Request X(A) → WAIT

↓

DEADLOCK

↓

Abort T2

↓

T1 gets X(B)

↓

T1 completes

Conclusion

Basic 2PL ensures that a transaction first acquires all required locks during the **growing phase** and releases locks during the **shrinking phase**. However, 2PL can cause **deadlocks**. A deadlock can be resolved by aborting or rolling back one of the transactions.

Q6. Wait-Die and Wound-Wait Deadlock Prevention

Given

Consider two transactions:

- **T1** → Timestamp = 10 → Older
- **T2** → Timestamp = 20 → Younger

Suppose **T2 holds a lock on data item A**, and **T1 requests the same lock**.

1. Wait-Die Scheme

Rule:

- Older transaction requests a lock held by younger → **Older transaction waits**.
- Younger transaction requests a lock held by older → **Younger transaction is aborted (dies)**.

Here:

T1 = Older

T2 = Younger

T1 requests A

T2 holds A

Since **T1 is older than T2**:

T1 → WAIT

T1 waits until T2 releases the lock.

Example

T2: Lock(A)

T1: Request Lock(A)

↓

T1 is older

↓

T1 waits

↓

T2 releases A

↓

T1 gets A

2. Wound-Wait Scheme

Rule:

- Older transaction requests a lock held by younger → **Younger transaction is aborted.**
- Younger transaction requests a lock held by older → **Younger transaction waits.**

Here:

T1 = Older

T2 = Younger

T2 holds A

T1 requests A

Since **T1 is older than T2**:

T1 wounds T2

Therefore, **T2 is aborted**, its locks are released, and T1 gets the lock.

Example

T2: Lock(A)

T1: Request Lock(A)

↓

T1 is older

↓

T2 is aborted

↓

T2 releases A

↓

T1 gets A

Comparison

Situation	Wait-Die	Wound-Wait
Older requests lock held by younger	Older waits	Younger aborts
Younger requests lock held by older	Younger aborts	Younger waits

Non-preemptive Preemptive

Wait-Die:

Younger $\rightarrow$ **Die**

Younger $\rightarrow$ **Wait**

Both **Wait-Die** and **Wound-Wait** use transaction timestamps to prevent deadlocks. They ensure that a circular waiting situation does not occur by either making a transaction **wait** or **abort** based on its age.

The question asks us to **apply the Immediate Update recovery protocol using transaction logs with checkpoints after a system failure.**

Consider the following log:

<T3, C, 300, 350>

Here:

- **T1** is committed.
- **T2** is uncommitted.
- **T3** is uncommitted.
- The system fails after the checkpoint.

Step 1: Identify Transactions

After the failure:

Committed Transactions:

T1

Uncommitted Transactions:

T2, T3

Step 2: REDO Committed Transactions

In **Immediate Update**, changes may be written to the database before the transaction commits.

Therefore, committed transactions are **REDO** operations.

T1:

A = 150

So, the change made by **T1 is retained**.

Step 3: UNDO Uncommitted Transactions

Transactions that did not commit before the failure must be **UNDO**.

For T3:

C: 350 → 300

For T2:

B: 250 → 200

Recovery Table

Transaction	Status	Recovery Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Uncommitted	UNDO

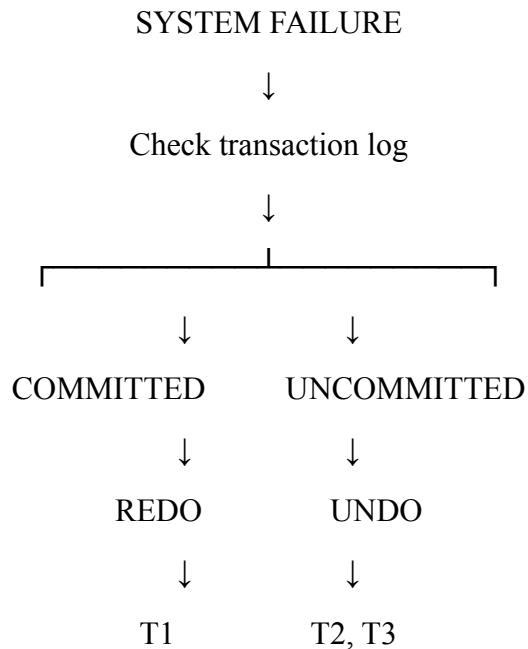
Final Database Values

A = 150

B = 200

C = 300

Simple Recovery Flow



Conclusion

In **Immediate Update Recovery**, database changes can occur before a transaction commits. After a system failure:

- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

The checkpoint helps the recovery process determine where to begin checking the log.

Q8. Deferred Update Recovery (NO-UNDO/REDO)

The question asks us to **apply the Deferred Update (NO-UNDO/REDO) recovery technique on a log containing committed and uncommitted transactions.**

Example Log

Consider the following log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

SYSTEM FAILURE

Step 1: Identify Transactions

From the log:

- **T1 → Committed**
- **T2 → Uncommitted**
- **T3 → Uncommitted**

Committed: T1

Uncommitted: T2, T3

Step 2: Apply Deferred Update

In **Deferred Update**, changes made by a transaction are **not written to the database until the transaction commits**.

Therefore:

- Committed transaction → **REDO**
- Uncommitted transaction → **No action**

Step 3: REDO T1

T1 has committed:

<T1, A, 100, 150>

Therefore:

A = 150

T1 is **REDO**.

Step 4: Ignore T2 and T3

T2 and T3 did not commit before the failure.

Therefore:

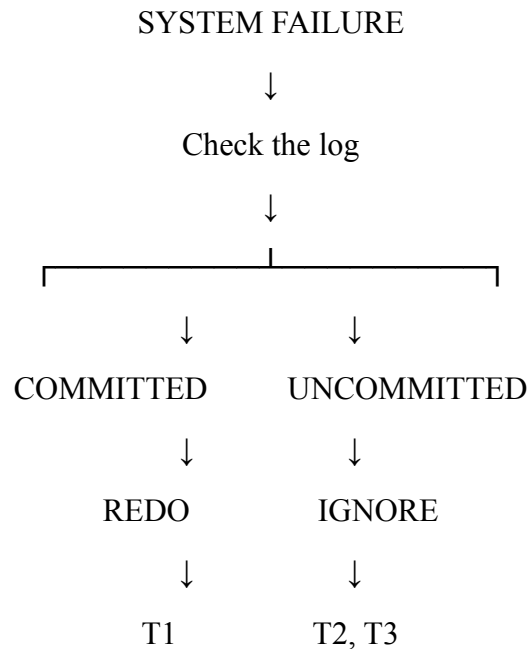
T2 → Ignore

T3 → Ignore

Recovery Table

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Uncommitted	Ignore

Recovery Process



Final Database

A = 150

B = 200

C = 300

The values of **B and C remain unchanged**, because T2 and T3 were not committed.

Conclusion

Deferred Update uses the NO-UNDO/REDO technique. Since database changes are deferred until commit, recovery only needs to **REDO committed transactions**. Uncommitted transactions are simply ignored, so **UNDO is not required**.

Q9. Concurrency Control Using Strict 2PL in a Banking System

The question asks us to **design a concurrency control mechanism using Strict 2PL** for a banking system where **T1 transfers funds and T2 reads the balance simultaneously**.

Given

Suppose:

- **T1** transfers ₹1,000 from Account A to Account B.
- **T2** reads the balance of Account A.
- Strict 2PL is used.

Transaction T1 – Transfer Funds

T1:

LOCK-X(A)

LOCK-X(B)

Read A

$A = A - 1000$

Read B

$B = B + 1000$

Write A

Write B

COMMIT

UNLOCK(A)

UNLOCK(B)

T1 obtains **exclusive locks** because it modifies both accounts.

Transaction T2 – Read Balance

T2:

LOCK-S(A)

Read A

COMMIT

UNLOCK(A)

T2 requires a **shared lock** because it only reads the balance.

Simultaneous Execution

Suppose T1 starts first:

T1: LOCK-X(A)

T1: LOCK-X(B)

T1: Update A and B

T2: LOCK-S(A) → WAIT

T1: COMMIT

T1: UNLOCK(A), UNLOCK(B)

T2: LOCK-S(A)

T2: Read A

T2: COMMIT

T2: UNLOCK(A)

Why Does T2 Wait?

T1 has an **exclusive lock on A**.

Therefore, T2 cannot read A until T1 completes:

T1 holds X(A)

↓

T2 requests S(A)

↓

WAIT

↓

T1 COMMIT

↓

T1 releases X(A)

↓

T2 gets S(A)

This prevents T2 from reading an intermediate or partially updated balance.

Strict 2PL Rules

1. Transaction obtains locks before accessing data.

2. **Shared lock (S)** is used for reading.
3. **Exclusive lock (X)** is used for writing.
4. Exclusive locks are held until **COMMIT or ROLLBACK**.
5. T2 waits while T1 holds the required exclusive lock.

Conclusion

Strict 2PL ensures that **T2 cannot read an inconsistent balance while T1 is transferring money**. T2 waits until T1 commits and releases its locks. Thus, the banking system maintains **consistency and isolation**.

Q10. Pseudocode for Heuristic Query Optimization

The question asks us to write pseudocode for query optimization using three heuristics: **push down selection, project early, and order joins by smaller intermediate results**.

Pseudocode

BEGIN

INPUT: SQL Query

1. Parse the SQL query into:

SELECT
FROM
WHERE
JOIN conditions

2. Push down selection:

For each WHERE condition
 Apply the condition to the related table
 as early as possible
End For

3. Project early:

For each table
 Keep only the columns required for

SELECT, WHERE and JOIN operations

End For

4. Order joins:

Identify all required joins

Estimate the intermediate result size

Perform the join producing the smallest
intermediate result first

5. Generate the optimized query execution plan

6. Execute the optimized plan

7. Return the final result

END

Simple Example

Suppose the query is:

SELECT S.Name, D.DeptName

FROM STUDENT S

JOIN DEPARTMENT D

ON S.DeptID = D.DeptID

WHERE S.Marks > 80;

The optimizer performs:

STUDENT

↓

Selection: Marks > 80

↓

Projection: Name, DeptID

↓

JOIN

/ \

/ \

DEPARTMENT Filtered STUDENT

↓

Projection:

DeptID, DeptName

↓

Final Result

Heuristics Used

1. **Push Down Selection**

Apply WHERE conditions as early as possible to reduce rows.

2. **Project Early**

Keep only required columns to reduce data size.

3. **Order Joins**

Perform the join that produces the **smallest intermediate result** first.

Conclusion

The heuristic optimizer improves query performance by **reducing the number of rows and columns processed before expensive join operations**. This results in a more efficient query execution plan.